JOB VACANCY DATA COLLECTOR BERBASIS CLI: UJIAN TENGAH SEMESTER AUTOMATION SYSTEMS

Disusun oleh:

Muhammad Iqbal Al Anshori (312310659)
Raihan Nurfadillah (312310134)
Rizkyawan Mu'adz Daffa (312310264)

Dosen Pengampu:

Anggi Muhammad Rifa'i, M.Kom

Mata Kuliah: Automation Systems

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PELITA BANGSA
BEKASI

2026

Daftar Isi

1 PENDAHULUAN **2**

1.1 Latar Belakang 2

1.2 Tujuan 2

2 METODOLOGI DAN IMPLEMENTASI **2**

2.1 Web Scraping (Chapter 13) 2

2.2 Regular Expressions (Chapter 9) 3

2.3 Reading & Writing Files (Chapter 10) 3

2.4 Organizing Files (Chapter 11) 3

2.5 Command Line Programs (Chapter 12) 3

2.6 Error Handling 3

3 HASIL DAN PEMBAHASAN **3**

3.1 Cara Penggunaan 3

3.2 Hasil Eksekusi Program 4

3.3 Organisasi File dan Backup 4

3.4 Hasil Pembersihan Data (CSV) 4

4 KESIMPULAN **5**

LAMPIRAN: Source Code **6**

1 PENDAHULUAN

1.1 Latar Belakang

Dalam era digital saat ini, mencari informasi lowongan pekerjaan secara manual dari berbagai situs pencari kerja memakan banyak waktu. Pencari kerja sering kali harus membuka satu per satu tautan untuk mendapatkan informasi detail seperti nama perusahaan, lokasi penempatan, dan tanggal posting. Data yang tersebar dan tidak terstruktur ini menyulitkan proses penyaringan lowongan yang relevan.

Oleh karena itu, diperlukan sebuah sistem otomatisasi yang dapat mengambil (*scraping*), mengekstrak, menyimpan, dan mengorganisasi data lowongan pekerjaan secara otomatis ke dalam format yang mudah dianalisis (CSV).

1.2 Tujuan

Tujuan dari pembuatan makalah dan proyek aplikasi ini adalah:

1. Mengimplementasikan teknik *Web Scraping* untuk mengambil data lowongan kerja dari halaman website publik.
2. Menerapkan *Regular Expressions* (Regex) untuk mengekstrak format lokasi dan memvalidasi format tanggal.
3. Mengotomatisasi pembuatan folder dan penyimpanan file menggunakan modul `pathlib` dan `csv`.
4. Membuat sistem *backup* data terkompresi ke dalam format ZIP.
5. Menyediakan antarmuka CLI yang interaktif beserta penanganan kesalahannya (*error handling*).

2 METODOLOGI DAN IMPLEMENTASI

Aplikasi ini dibangun menggunakan Python 3 dan mengintegrasikan materi dari Chapter 9 hingga Chapter 13 (Automate the Boring Stuff with Python).

2.1 Web Scraping (Chapter 13)

Kami menggunakan *library* `requests` untuk mengambil halaman HTML dari `https://realpython.github.io/fake-jobs/` dan `BeautifulSoup` untuk mem-*parsing* HTML tersebut. Program diatur untuk mengambil minimal 30 data *record* yang mencakup Judul Pekerjaan, Perusahaan, Lokasi, Tanggal Posting, dan URL.

2.2 Regular Expressions (Chapter 9)

Data mentah yang diambil dari website diproses menggunakan dua pola Regex:

1. **Regex 1 (Ekstraksi Lokasi):** Pola r"([A-Za-z\s]+),\s*([A-Z]{2})" digunakan untuk mengambil format ”Kota, Negara Bagian” dari teks alamat yang berantakan.
2. **Regex 2 (Validasi Tanggal):** Pola r"\^d{4}-\d{2}-\d{2}$" digunakan untuk memastikan format tanggal posting adalah YYYY-MM-DD.

2.3 Reading & Writing Files (Chapter 10)

Hasil data yang sudah bersih disimpan ke dalam dua format. Pertama, disimpan sebagai file output/hasil_scraping.csv menggunakan modul csv. Kedua, disimpan sebagai laporan teks di output/laporan.txt dan file mentah di data/raw_data.txt.

2.4 Organizing Files (Chapter 11)

Pembuatan folder (data/, output/, backup/) dilakukan secara otomatis menggunakan modul pathlib dengan perintah Path().mkdir(parents=True,exist_ok=True). Fitur *backup* membuat file backup_data.zip menggunakan *library* zipfile.

2.5 Command Line Programs (Chapter 12)

Aplikasi dikendalikan melalui CLI menggunakan *library* argparse. Argumen yang disediakan meliputi --url, --keyword, --limit, --output, dan --backup.

2.6 Error Handling

Program dilengkapi dengan try-except untuk menangani kegagalan koneksi internet atau URL salah (requests.exceptions.RequestException). Jika data kosong (ifnotdata_hasil), program memberikan peringatan rapi tanpa *crash* dan menampilkan ringkasan di terminal.

3 HASIL DAN PEMBAHASAN

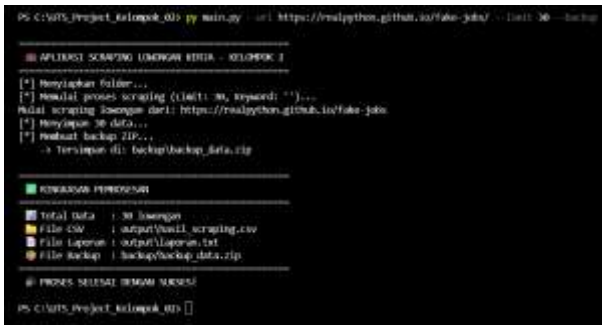
3.1 Cara Penggunaan

Aplikasi dijalankan melalui terminal (Command Prompt / VS Code) dengan perintah:

```
python main.py --url https://realpython.github.io/fake-jobs/ --limit 30
    --backup
```

3.2 Hasil Eksekusi Program

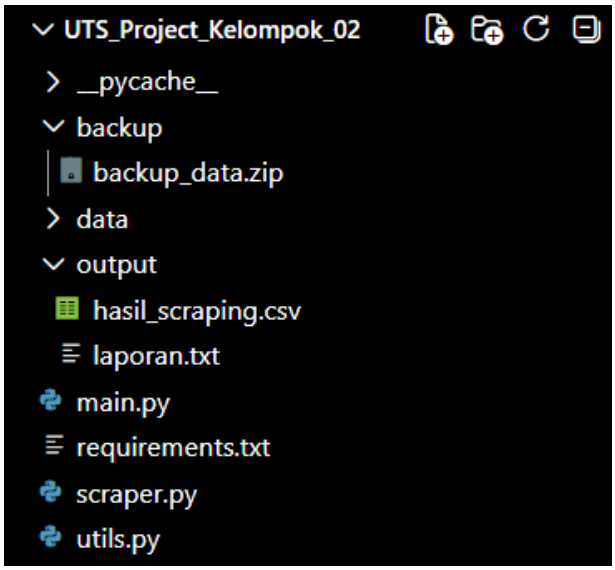
Setelah program dijalankan, terminal menampilkan proses *scraping* hingga ringkasan pemrosesan (Total 30 lowongan).



Gambar 1: Output terminal VS Code saat program berhasil dijalankan

3.3 Organisasi File dan Backup

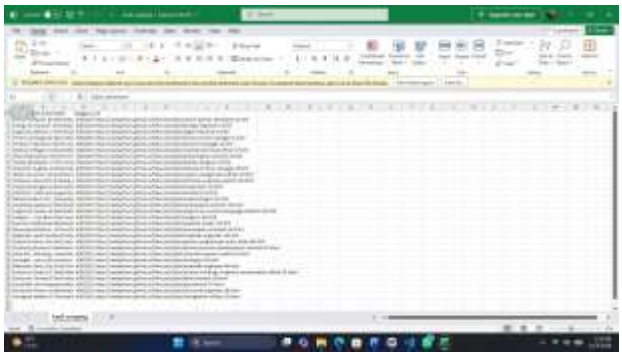
Program secara otomatis membuat folder `data`, `output`, dan `backup`. File CSV, TXT, dan ZIP berhasil dibuat sesuai dengan struktur yang diminta.



Gambar 2: Struktur folder dan file yang dibuat secara otomatis

3.4 Hasil Pembersihan Data (CSV)

File `hasil_scraping.csv` dibuka di Microsoft Excel. Regex berhasil mengekstrak lokasi (Kota, Negara Bagian) dan memvalidasi tanggal posting dengan format yang seragam.



Gambar 3: Hasil ekstraksi data CSV yang dibuka di Microsoft Excel

4 KESIMPULAN

Berdasarkan hasil perancangan dan pengujian, aplikasi otomatisasi berbasis Python CLI berhasil dibangun. Program mampu mengotomatisasi pengambilan data lowongan kerja, membersihkan format lokasi dan tanggal dengan *Regular Expressions*, mengelola penyimpanan file ke format CSV/TXT, serta menyediakan fitur *backup* ZIP. Aplikasi telah dilengkapi *error handling* sehingga memenuhi seluruh kriteria UTS Automation Systems.

LAMPIRAN: Source Code

1. requirements.txt

```
requests
beautifulsoup4
```

2. main.py

```
import argparse
import sys
from pathlib import Path
import requests
from scraper import scrape_lowongan
from utils import buat_folder, simpan_csv, buat_laporan, buat_backup_zip

def main():
    parser = argparse.ArgumentParser(description="Aplikasi Scraping Lowongan Kerja (UTS Automation Systems)")
    parser.add_argument("--url", required=True, help="URL sumber data (contoh : https :// realpython . github . io/ fake - jobs /)")
    parser.add_argument("--keyword", default="", help="Kata kunci untuk memfilter judul lowongan")
    parser.add_argument("--limit", type=int, default=30, help="Jumlah maksimal data yang diambil")
    parser.add_argument("--output", default="output", help="Nama folder untuk menyimpan hasil")
    parser.add_argument("--backup", action="store_true", help="Tambahkan flag ini untuk membuat file ZIP backup")

    args = parser.parse_args()

    print("\n" + "="*50)
    print("       APLIKASI SCRAPING LOWONGAN KERJA - KELOMPOK 2")
    print("="*50)

    print("[*] Menyiapkan folder...")
    buat_folder("data")
    buat_folder(args.output)
    buat_folder("backup")

    url_bersih = args.url.rstrip('/')

    print(f"[*] Memulai proses scraping (Limit: {args.limit}, Keyword: '{args.keyword}')...")
```

```
    try:
        data_hasil = scrape_lowongan(url_bersih, keyword=args.keyword,
    limit=args.limit)
    except Exception as e:
        print(f"\n[   ] ERROR tidak terduga: {e}")
        sys.exit(1)

    if not data_hasil:
        print("\n" + "!"*50)
        print("          PERINGATAN: DATA KOSONG")
        print(" Tidak ada lowongan yang cocok dengan kriteria.")
        print("!"*50 + "\n")
        sys.exit(1)

    print(f"[*] Menyimpan {len(data_hasil)} data...")
    path_csv = Path(args.output) / "hasil_scraping.csv"
    path_txt = Path(args.output) / "laporan.txt"

    fieldnames = ["judul_pekerjaan", "perusahaan", "lokasi", "
    tanggal_posting", "url"]
    simpan_csv(data_hasil, path_csv, fieldnames)
    buat_laporan(data_hasil, path_txt)

    with open("data/raw_data.txt", "w", encoding="utf-8") as f:
        for item in data_hasil:
            f.write(str(item) + "\n")

    if args.backup:
        print("[*] Membuat backup ZIP...")
        path_zip = Path("backup") / "backup_data.zip"
        buat_backup_zip(args.output, path_zip)
        print(f"    -> Tersimpan di: {path_zip}")

    print("\n" + "="*50)
    print("      RINGKASAN PEMROSESAN")
    print("="*50)
    print(f"      Total Data   : {len(data_hasil)} lowongan")
    print(f"      File CSV     : {path_csv}")
    print(f"      File Laporan : {path_txt}")
    if args.backup:
        print(f"      File Backup  : backup/backup_data.zip")
    print("="*50)
    print("       PROSES SELESAI DENGAN SUKSES!\n")

if __name__ == "__main__":
    main()
```

3. scraper.py

```python
import re
import requests
from bs4 import BeautifulSoup

def ekstrak_lokasi(teks_lokasi):
    # REGEX 1: Mengambil format "Kota, Negara Bagian" (contoh: "Stewartbury, AA")
    pola = r"([A-Za-z\s]+),\s*([A-Z]{2})"
    hasil = re.search(pola, teks_lokasi)
    if hasil:
        return hasil.group(0).strip()
    return teks_lokasi.strip()

def validasi_tanggal(teks_tanggal):
    # REGEX 2: Memastikan format tanggal YYYY-MM-DD
    pola = r"^\d{4}-\d{2}-\d{2}$"
    if re.fullmatch(pola, teks_tanggal.strip()):
        return teks_tanggal.strip()
    return "Tanggal tidak valid"

def scrape_lowongan(url_dasar, keyword="", limit=30):
    hasil_data = []
    print(f"Mulai scraping lowongan dari: {url_dasar}")

    try:
        response = requests.get(url_dasar, timeout=10)
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        print(f"[!] Gagal koneksi: {e}")
        return hasil_data

    soup = BeautifulSoup(response.text, "html.parser")
    semua_kartu = soup.select("div.card-content")

    for kartu in semua_kartu:
        judul = kartu.select_one("h2.title").text.strip()

        if keyword and keyword.lower() not in judul.lower():
            continue

        perusahaan = kartu.select_one("h3.company").text.strip()
        lokasi_mentah = kartu.select_one("p.location").text
        lokasi_bersih = ekstrak_lokasi(lokasi_mentah)

        tanggal_mentah = kartu.select_one("time")["datetime"]
```

```
        tanggal_valid = validasi_tanggal(tanggal_mentah)

        link = kartu.find_all("a")[-1]["href"]

        data_lowongan = {
            "judul_pekerjaan": judul,
            "perusahaan": perusahaan,
            "lokasi": lokasi_bersih,
            "tanggal_posting": tanggal_valid,
            "url": link
        }
        hasil_data.append(data_lowongan)

        if len(hasil_data) >= limit:
            break

    return hasil_data
```

4. utils.py

```
import csv
import zipfile
from pathlib import Path

def buat_folder(nama_folder):
    Path(nama_folder).mkdir(parents=True, exist_ok=True)

def simpan_csv(data, path_file, fieldnames):
    with open(path_file, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(data)

def buat_laporan(data, path_file):
    with open(path_file, "w", encoding="utf-8") as f:
        f.write(f"=== LAPORAN HASIL SCRAPING ===\n")
        f.write(f"Total record ditemukan: {len(data)}\n\n")
        for item in data:
            f.write(str(item) + "\n")

def buat_backup_zip(folder_output, path_zip):
    with zipfile.ZipFile(path_zip, "w", zipfile.ZIP_DEFLATED) as zf:
        for file in Path(folder_output).rglob("*"):
            if file.is_file():
                zf.write(file, file.relative_to(folder_output))
```